

AI Collaboration Log

CSC322 Group E — CalcEngine

Every significant use of an AI assistant on this project, in date order. For each: what I asked, what came back, what I changed, and why.

The rule I worked to is the one in the brief: *an engineer who ships code is responsible for that code, no matter who or what wrote the first draft*. In practice that meant a standing habit — **no AI-produced code entered the repository until one of us could state, out loud, what would break if it were wrong**. Several entries below are things I did not ship.

Tools used: **Claude (Opus)** for design discussion and drafting, **GitHub Copilot** for line completion inside the editor, **ChatGPT** for the critique exercise (transcript in [critique/](#)).

Where an entry says "rejected", the code is not in the repository. Where it says "reworked", the shape survived and the substance changed.

1 · 23 August · Claude · Scoping the whole engine

Asked. For an architecture for a spreadsheet calculation engine meeting the brief, before I wrote anything.

Got. A sensible five-box pipeline — parser, expression tree, dependency graph, evaluator, notification — which is essentially the architecture in [design-portfolio.md](#) §2. It also proposed storing cells as `Dictionary<string, Cell>` keyed by `"Sheet1!B2"`.

Changed. Kept the pipeline; **rejected the string keys**. A string key means hashing a string on every graph lookup, and the graph is walked once per dependent per keystroke. I took integer identifiers instead, interned per (sheet, column, row). This was the first decision of the project and it set the shape of `DependencyGraph`.

Why it mattered. The 50 ms target is not tight if the traversal is array indexing and is genuinely tight if it is string hashing. I did not measure the rejected version, which is a gap — see the reflection.

2 · 23 August · Claude · The grammar, first draft

Asked. For an ANTLR 4 grammar for a spreadsheet formula language with the eight required functions.

Got. A workable grammar. Two problems I caught by reading it:

- It gave `^` higher precedence than unary minus, so `--2^2` would be `-4`. Excel says `4`. I flipped the alternative order and wrote `Negation_BindsTighterThanPower_LikeExcel` so the decision is a test rather than folklore ([grammar.md](#) §4.1).
- It lexed sheet names as a bare `IDENTIFIER` followed by `'!'`, which is ambiguous with a cell reference in a lexer that does not backtrack. I made the `'!'` part of the `SHEET_QUALIFIER` token so maximal munch decides it.

Also rejected. Its suggestion of whole-column ranges (`A:A`). One edge would then stand for 1,048,576 cells, which defeats the range index I had already decided on. That exclusion is documented, not silent.

3 · 23 August · Copilot · `CellAddress.ColumnName`

Asked. Nothing — completion offered a bijective base-26 conversion as I typed the signature.

Got. Correct-looking code with an off-by-one: it treated the conversion as ordinary base-26, so column 27 came out as `BA` rather than `AA`.

Changed. Fixed the `remaining--` placement. **Process change:** this is why `ColumnName_UsesBijectiveBase26` tests 1, 26, 27, 52, 53, 702, 703 and 16384 rather than "a few values" — those are exactly the boundaries the wrong algorithm gets wrong.

4 · 23 August · Claude · Dependency graph, first design (rejected)

Asked. For a dependency graph supporting fast edge insertion and removal, cycle detection and topological ordering.

Got. `Dictionary<CellRef, HashSet<CellRef>>` in both directions, and a `Recalculate()` that rebuilt a topological order of **the entire workbook** on every change and re-evaluated every formula in it.

Rejected, and this is the entry I would point to first. It is correct. It is also `0(all formulas)` per keystroke, which for the brief's 100,000-cell workbook means a full recalculation every time a mark is typed — roughly 75 ms in our final build, so it would even have *passed* the 50 ms target on a fast day and failed on a slow one, which is worse than failing outright.

Built instead. A DFS from the edited cell over dependent edges, producing a topological order of the affected subgraph only. Cost follows the edit. `OnlyAffectedCellsAreRecomputed` asserts on `CellsEvaluated`, not on values, precisely so that a regression to the rejected design fails the test rather than merely slowing things down.

5 · 23 August · Claude · Ranges in the dependency graph

Asked. How to make `=SUM(B2:B45)` depend on its range.

Got. "Expand the range and add an edge per cell." When pushed on `=SUM(A1:A100000)`, it suggested capping range size.

Rejected both. The cost argument is real but secondary. The fatal objection is that typing into a cell that was *empty when the formula was written* must still trigger the formula, and an empty cell has no edge. A cap does not fix that; it just makes the failure size-dependent.

Built instead. The 64×64 spatial index in `RangeDependencyIndex`, with an always-scanned fallback list for pathologically large ranges. `WritingIntoAPreviouslyEmptyCellOfARangeStillTriggersIt` is the test that names the reason.

Honest note. The AI did not think of this; I found the empty-cell case by hand-tracing what happens when a lecturer fills in a mark for a student who was absent at export time. I then asked the AI to critique the block-index design, and it correctly pointed out the unbounded-range case, which is where the `MaxBlocksPerRange` fallback comes from. That exchange is a fair example of where the tool was genuinely useful: not at inventing the structure, but at attacking it once it existed.

6 · 23 August · Claude · Cycle detection

Asked. For circular-reference detection reporting the exact cycle.

Got. A recursive DFS with a `HashSet<CellRef> visiting` and a `List` path, throwing `CircularReferenceException` on a back edge.

Reworked substantially. Three changes:

1. **No exception.** The brief says report the cycle to the client; a workbook with two independent cycles must report both, and an exception reports one and abandons the recalculation. Cycles became part of `CalculationResult`.
 2. **No recursion.** A 20,000-cell import chain overflows the stack, and `StackOverflowException` cannot be caught — the client loses the process, not the operation. Rewritten with an explicit frame stack; `ALongCycleDoesNotOverflowTheStack` holds the line at 20,000.
 3. **Canonical path.** The draft reported the cycle starting wherever the search entered it, so the same broken workbook produced different messages depending on typing order. I reverse the stack slice (the search travels "is read by"; users think "refers to") and rotate to the smallest address.
-

7 · 23 August · Claude · Error messages from ANTLR

Asked. How to turn ANTLR's `mismatched input ')' expecting {NUMBER, STRING, ...}` into something a lecturer can act on.

Got. A `BaseErrorListener` that reformats the message. Useful, and it is the basis of `DescriptiveErrorListener`.

Added ourselves. The listener alone still produced a cascade for the single most common mistake — a missing `)`. ANTLR's recovery invents tokens and then complains about the consequences, pointing at the wrong place. I added `TokenPreValidator`: one linear pass over the flat token list that catches stray characters, unterminated text and bracket imbalance *before* the parser runs, so the engine can say "the bracket opened at column 5 was never closed" — naming the bracket the user has to go and fix.

Evidence it was worth it. `SyntaxErrorMessageTests` pins the exact wording and column of every message. Two of our own expectations in that file were wrong when first written (I mis-counted a column); the commit that fixed them says so.

8 · 23 August · ChatGPT · The critique exercise

Asked for a complete implementation of the dependency-graph module and reviewed it as a senior engineer. Full transcript and two-page review in `critique/`. Summary: correct on the happy path,

`O(V+E)` over the whole workbook per edit, recursive cycle detection, and a representation-invariant break on formula edit that leaks stale reverse edges.

9 · 23 August · Claude · `IF` and laziness

Asked. For the standard function library.

Got. Implementations taking `IReadOnlyList<CellValue>` — arguments already evaluated.

Rejected the signature, kept the bodies. With eager arguments, `=IF(A1=0,"n/a",B1/A1)` returns `#DIV/0!` for exactly the input it was written to guard against. `IFunction` takes `FunctionArguments`, which holds the *unevaluated* expressions and evaluates on first access with caching. `If_DoesNotEvaluateTheBranchItDoesNotTake` is the test.

Cost of the change. Every function became slightly more verbose (`arguments.Number(0)` rather than `values[0]`). I think that is the right trade; a reviewer might reasonably disagree and ask for both interfaces.

10 · 23 August · Claude · `SUM` and text

Asked. Why our `=SUM(A1:A9)` disagreed with Excel on a column containing the word "absent".

Got. A clear explanation of Excel's asymmetry: values read *through a reference* are data and are skipped when of the wrong type; values written *into the formula* are instructions and are coerced. `=SUM(A1)` with `A1 = "3"` is 0; `=SUM("3")` is 3.

Changed. Added `ArgumentValue.FromReference` and the rule in `Aggregation.CollectNumbers`. This is the entry where the tool was most straightforwardly valuable: it is documented Excel behaviour that none of us knew, and getting it wrong would silently change the totals of every sheet that records absences as text.

Verified independently, in Excel, before implementing. I did not take it on trust — an earlier answer in the same conversation had confidently told us that `COUNT` counts booleans inside ranges, which it does not.

11 · 23 August · Copilot · `ROUND` with negative digits

Got. `Math.Round(value * Math.Pow(10, digits), MidpointRounding.AwayFromZero) / Math.Pow(10, digits)`.

Reworked. For `digits = -2` this multiplies by `0.01`, which is not exactly representable: `ROUND(1250, -2)` came out as `1299.9999999999998`. Rewritten to divide by `100` and multiply back for negative digit counts. The comment in `StandardLibrary.Round` explains it so nobody "simplifies" it back.

12 · 23 August · Claude · Find & Replace inside formulas

Asked. For a Find & Replace across a workbook.

Got. A clean implementation doing `content.Replace(find, replace)` on every matching cell, formulas included.

Accepted as the default, and then attacked. I asked what it does to `=SUM(B2:B9)` when replacing "B2" with "B3". It answered, correctly, that the range is rewritten — and added "which is usually what the user wants". For a *sheet rename* it is. For a course code that happens to look like a reference it is a silent corruption of every total on the sheet.

Added. `FormulaHandling.TextLiteralsOnly`, which rewrites only inside quoted text using `TextLiteralFinder` and the `SourceSpan` of each literal; and `ValidateFormulas`, which re-parses the rewritten formula and refuses with the parser's own message rather than storing something broken. Both modes are pinned by adjacent tests, so the difference between them is documented behaviour.

What this cost. `SourceSpan` exists on every expression node because of this feature. That is a real cost imposed on the core ADT by a feature, and I would defend it: spans also improve error reporting and would be needed by any future structural edit.

13 · 23 August · Claude · Duplicate keys

Asked. For duplicate detection over a range with selectable key columns.

Got. A dictionary-based single pass — the right algorithm — with the key built as `string.Join("|", values.Select(v => v.ToString()))`.

Rejected the key. Two collisions:

- `5` and `"5"` produce the same string, so an imported column of text marks appears to duplicate a typed one.
- `{"a", "b"}` and `{"a|b"}` produce the same string.

Built instead. A one-character type tag plus a length prefix per field. `ANumberIsNotTheSameAsTheTextThatLooksLikeIt` is the test for the first; the length prefix is argued in `adt-specifications.md` §10.

Where the AI was right and I initially were not. I proposed comparing `CellValue`s structurally with a tuple key instead. It pointed out that the case-insensitive and whitespace-trimming options make equality *configuration-dependent*, so the comparison has to be normalised into a key anyway. It was correct.

14 · 23 August · Claude · Remove Duplicates and row deletion

Asked. For `ShiftUp` removal that behaves like Excel's Remove Duplicates.

Got. Content-shifting code, plus — unprompted and correctly — a warning that relative references in moved formulas would not follow them.

Decision, ours. I considered building the reference rewriter. I did not, because it is a whole-workbook formula rewrite with its own ADT and its own test suite, and a partially correct version fails *silently*. Instead `ShiftUp` refuses when a formula would move, names the cells, and offers

`AllowMovingFormulas` for the data-only case. The reasoning is in `design-portfolio.md` §7.2 and repeated in the reflection as the thing I would build next.

15 · 23 August · Claude · Blazor grid

Asked. For a scrollable grid component with in-cell editing.

Got. A working component. It kept an edit buffer and synchronised it with `if (_lastSelection != Session.Selected.Address) { ... }`.

Bug, found by running it, not by reading it. `default(CellAddress)` is `A1` by design (see `adt-specifications.md` §2). On the first render `_lastSelection` is `default` and the selection is `A1`, so the test says "unchanged", the buffer is never seeded, and the first click elsewhere commits an empty string over `A1` — deleting the header of the sample sheet.

Changed. An explicit `_seeded` flag. The comment names the trap.

Process note. I only saw this because the browser smoke test (`tools/gui-smoke.js`) screenshots the sheet, and a reviewer noticed the missing header in the image. Neither the AI nor our reading caught it. It is the single best argument in this project for running the thing you built.

16 · 23 August · Claude · Benchmark harness

Asked. For a harness measuring the two published targets.

Got. A reasonable harness that reported the **median**.

Changed. A target is met when the *worst* run is inside the budget. Reporting a median that fits while the tail does not is a way of not answering the question. `BenchmarkResult.Passed` compares `MaxMs`.

Second bug, found by the harness. The first run threw: "expected at least 500 dependents to recompute, got 0". Two separate causes, both real:

- `Workbook` consulted the *constructor option* rather than the mutable `AutomaticCalculation` property on every edit, so a workbook created for a bulk import could never be switched back to automatic calculation. Fixed, with `CalculationCanBeSwitchedBackOnAfterABulkLoad`, whose comment says where it came from.
 - Our own benchmark wrote the value the cell already held, which is correctly a no-op — so it was measuring nothing. Fixed in the harness.
-

17 · 23 August · Claude · Long-cycle memory blow-up

Context. `ALongCycleDoesNotOverflowTheStack` (20,000 cells in one ring) failed with `OutOfMemoryException` after 76 seconds.

Asked. For help reading the stack trace.

Got. The right diagnosis immediately: the error *message* for a cycle was being interpolated once per member cell, and each message contains the whole path. 20,000 cells × a 200 KB path is 4 GB.

Changed. One `ErrorValue` per ring, shared by every cell in it; and `CircularReference.ToString()` abbreviates beyond 12 cells while `Path` keeps them all. The whole test suite went from 1 minute 26 seconds to 1 second.

Reflection on the tool. This is what it is best at: reading a stack trace and a code path faster than a tired human at 11 p.m. It is not what it is best at when the question is "should this exist at all".

18 · 23 August · Copilot · Test data

Generated the sample results sheet — twelve students, Nigerian names, plausible CA/exam splits, two deliberate duplicate registrations. Reviewed and kept. No correctness content; recorded for completeness because the brief asks for *every* significant use, and inventing plausible test data is a real use.

19 · 23 August · Claude · Documentation review

Asked. To review this portfolio for claims not supported by the code.

Got. Three catches, all fair:

- I had written that the engine "detects circular references in $O(1)$ ". It is $O(V+E)$ over the affected subgraph. Corrected.
- I had claimed the range index "eliminates" range-dependency cost. It bounds it. Corrected.
- I had described `ShiftUp` as "Excel-compatible" without qualifying the reference behaviour. Now stated explicitly as a limitation.

What I did not accept. It proposed rewriting the ADT specifications in a more formal notation (Z-style schemas). I kept the prose-plus-predicate style, because the specifications also live as XML documentation on the types, and a notation nobody reads in IntelliSense is a specification that drifts.

20 · 23 August · Whole team · Line-by-line read-through

Not an AI session: the four of us read the entire `CalcEngine.Core` source aloud, in one sitting, with the rule that whoever could not explain a line owned rewriting it. Four things came out of it:

- `FormulaPrinter.Wrap` had a parameter (`parent`) that was only meaningful for binary nodes. Kept, documented.
 - Nobody could explain `_idByKey` versus `Sheet.IdByAddress` without looking. I added the comment in `Workbook.Key` that says the first is the global (sheet, column, row) intern table and the second is the per-sheet view.
 - `AndOrFunction` silently skipped text inside references. It is Excel's rule but it was undocumented; it now says so.
 - One of us could not explain why `ErrorValue` compares by `Kind` alone. Now `adt-specifications.md` §4.1 does, and so can they.
-

What I would tell next year's group

The tool is a fast, confident, occasionally wrong colleague. It was genuinely useful three times over — Excel's `SUM` asymmetry (entry 10), the `OutOfMemoryException` diagnosis (entry 17), and attacking a design once it existed (entry 5). It was actively harmful twice: the whole-workbook recalculate (entry 4) and the naive duplicate key (entry 13) are both *plausible*, both *correct on small inputs*, and both wrong in the way this project is graded on.

The pattern is consistent. It is good at the shape of a solution and bad at the constraint that makes the problem hard. It proposed the right five-box architecture in one go and then, in the same conversation, an implementation that missed the performance target by two orders of magnitude — because the target was in our head, not in the prompt.

So: state the constraint in the prompt, and test the constraint, not the output.

`OnlyAffectedCellsAreRecomputed` asserts on `CellsEvaluated` rather than on values, so a regression to the rejected design fails a test instead of merely getting slower. That test is the single most useful thing in this repository, and it exists because of entry 4.